

1. [2p] Dado el siguiente código Java,

- a) Dibujar un diagrama de clases que muestre todas las clases/interfaces involucradas (salvo Object) y sus relaciones. No es necesario incluir los atributos y métodos, a menos que sean relevantes para mostrar las relaciones entre las clases.
- b) Justificar: ¿Las clases corresponden a la capa *modelo* o *presentación*?

Personaje.java

```
1 public abstract class Personaje {
2     protected Vector2 posicion = new Vector2(0, 0);
3     protected final String nombre;
4
5     protected Personaje(String nombre) {
6         this.nombre = nombre;
7     }
8
9     public Vector2 getPosicion() {
10         return posicion;
11     }
12
13     public void setPosicion(Vector2 nuevaPosicion) {
14         this.posicion = nuevaPosicion;
15     }
16
17     public abstract void actualizar(Mundo mundo);
18 }
```

Mundo.java

```
1 public interface Mundo {
2     public void agregarPersonaje(Personaje personaje);
3     public Personaje obtenerPersonaje(String nombre);
4     public void eliminarPersonaje(String nombre);
5 }
```

Mario.java

```
1 public class Mario extends Personaje {
2     private Entrada entrada;
3
4     public Mario(Entrada entrada) {
5         super("Mario");
6         this.entrada = entrada;
7     }
8
9     @Override public void actualizar(Mundo mundo) {
10         if (entrada.teclaPresionada(IZQUIERDA)) {
11             posicion = posicion.mover(-1, 0);
12         }
13         // ...
14     }
15 }
```

Koopa.java

```
1 public class Koopa extends Personaje {
2     public Koopa() { super("Koopa"); }
3
4     @Override public void actualizar(Mundo mundo) {
5         if (mundo.getPersonaje("Mario").getPosicion().x <
6             posicion.x) {
7             posicion = posicion.mover(-1, 0);
8         }
9         // ...
10     }
11 }
```

2. [1p] Considerar el siguiente código. Identificar un principio de diseño que se viola, indicar cuál es y justificar. Proponer una solución para evitar la violación del principio (no es necesario escribir el código).

```
1 public class Archivo {
2     void abrir() { ... }
3     void cerrar() { ... }
4     byte[] leer(int n) { ... }
5     void escribir(byte[] datos) { ... }
6 }
7
8 public class ArchivoSoloLectura extends Archivo {
9     @Override
10     void escribir(byte[] datos) {
11         throw new UnsupportedOperationException("El archivo es de solo lectura");
12     }
13 }
```

3. [2p] Escribir en Prolog la regla ultimo/2 que relaciona una lista con su último elemento. Por ejemplo:

?- ultimo([a, b, c, d], X).

X = d.

4. [2p] Dado el siguiente código Clojure, justificar:

- ¿Cuál o cuáles funciones definidas en el código (con nombre o anónimas) son puras/impuras?
- ¿Cuál o cuáles funciones definidas en el código (con nombre o anónimas) son de orden superior?

```
(defn crear-agenda [] {})  
  
(defn agregar-contacto [agenda nombre numero]  
  (assoc agenda nombre numero))  
  
(defn obtener-numero [agenda nombre]  
  (get agenda nombre))  
  
(defn recorrer [agenda f]  
  (doseq [[nombre numero] agenda]  
    (f nombre numero)))  
  
(defn imprimir-agenda [agenda]  
  (recorrer agenda  
    (fn [nombre numero]  
      (println (str nombre ": " numero))))))  
  
(defn main []  
  (let [agenda (crear-agenda)  
        agenda (agregar-contacto agenda "Alan" "1234")  
        agenda (agregar-contacto agenda "Barbara" "5678")]  
    (imprimir-agenda agenda)))
```

5. [2p] Dada la siguiente expresión lambda,

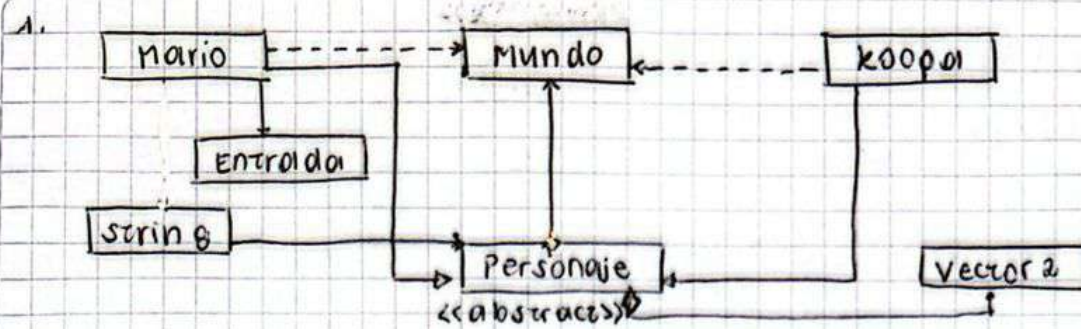
- Identificar las variables libres y ligadas.
- Mostrar los pasos de reducción usando orden normal (β -redex más externa desde la izquierda) hasta llegar a la forma normal (si existe). Aplicar reducciones α según sea necesario para evitar conflictos de nombres.

$(\lambda x. (\lambda y. (x \ y)) \ w)$

6. [1p] Dado el siguiente código JavaFX:

- Dibujar el árbol correspondiente al Scene Graph.
- Explicar brevemente el funcionamiento del programa, en términos de programación GUI y orientada a eventos. ¿Qué sucede cuando el usuario interactúa con la interfaz? ¿Cómo se logra este comportamiento?

```
public class App extends Application {  
  @Override  
  public void start(Stage stage) throws Exception {  
    Label lbl = new Label("Nombre:");  
    TextField nombre = new TextField();  
  
    ListView<String> inscriptos = new ListView<>();  
  
    Button btnAgregar = new Button("Agregar");  
    btnAgregar.setOnAction(_ -> {  
      inscriptos.getItems().add(nombre.getText());  
    });  
  
    Button btnQuitar = new Button("Quitar");  
    btnQuitar.disableProperty().bind(  
      Bindings.isEmpty(  
        inscriptos.getSelectionModel().getSelectedItem()  
      ))  
    );  
    btnQuitar.setOnAction(_ -> inscriptos.getItems().remove(  
      inscriptos.getSelectionModel().getSelectedIndex()  
    ));  
  
    stage.setScene(new Scene(new VBox(  
      new HBox(lbl, nombre, btnAgregar),  
      inscriptos,  
      btnQuitar  
    )));  
    stage.show();  
  }  
}
```



b) capa modelo: Lógica de entidades.

reemplazar a la clase

2. se viola LSP ya que una instancia de la subclase no puede superclase sin romper el comportamiento esperado.

3.

?-ultimo([a,b,c,d],x)

como el predicado involucra una lista, se plantea recursivamente.

ultimo([x],x). % hecho

ultimo([_]τ,x):- ultimo(τ,x)

4.

a) La función crear-agenda es pura ya que tiene comportamiento determinístico.

La función agregar-contacts es pura ya que utiliza internamente la función Assoc. Esta última, al agregársele una nueva clave al diccionario, crea uno nuevo con los mismos elementos que el anterior y el nuevo asociado. Esto mantiene la inmutabilidad y, por consecuencia, lograr que la función sea pura.

La función imprimir-agenda es impura ya que produce efectos colaterales al imprimir por pantalla las claves y valores del diccionario. "Produce un cambio en el exterior".

Por este mismo motivo, la función main también será impura ya que utiliza imprimir-agenda

Obtener-numero también es pura ya que es determinística.

La función recorrer es pura pero depende de F, por lo que dependerá de cómo sea esta.

b) La única función de orden superior presente en el código es recorrer.

$$5. (((\lambda x. (xx)) (\lambda y. (yx))) w)$$

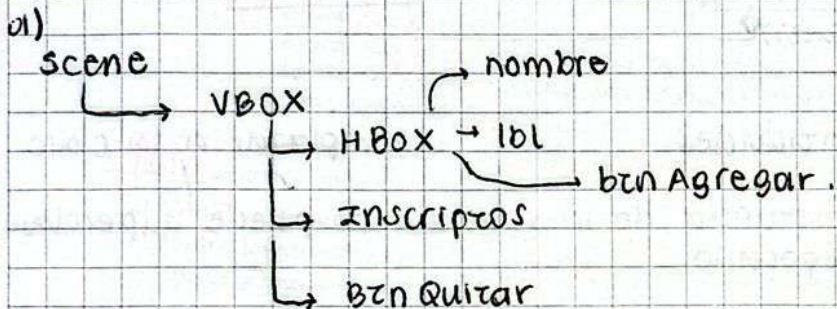
$$((\lambda x. xx) (\lambda y. yx)) w$$

$$= \beta ((\lambda y. yx) (\lambda y. yx)) w$$

$$= \beta ((\lambda y. yx) x) w$$

$$= \beta \quad xxw$$

6.



b) El programa tiene una label y un textfield con el cual el usuario describe funcionamiento hablando de eventos y bindings!